Data Mining 1 Project

Giulia Fabiani Camilla Maffeis

A.Y. 2024/2025

CONTENTS

1 DATA UNDERSTANDING AND PREPARATION

The goal of the project is to analyze the IMDb Dataset, which contains data about movies, TV shows, and other forms of visual entertainment, along with their ratings. Each record includes key information about the title, as well as insights into critical aspects such as awards and reviews, as well as statistical ratings and other metadata. The dataset is updated as of September 1, 2024.

1.1 Data Semantics and First Global Exploration of the Dataset

The IMDb Dataset contains 16431 records and 23 attributes, of which 10 are categorical and 13 numeric. They are listed in Tables 4 and 5. Among the **numerical attributes**, most of them are discrete and ratio-scaled, as they represent counts, with *startYear* and *endYear* being interval values. Only *runtimeMinutes* can be classified as a continuous attribute, as it represents a time duration, even though it only assumes integer values in the dataset. Among the **categorical attributes**, *canHaveEpisodes*, *isRatable* and *isAdult* are **binary**, *rating*, *worstRating* and *bestRating* are **ordinal**, while the rest are all **nominal**.

Looking at the records' data types, we can observe some syntactic inaccuracies: *isAdult* should be converted from integer to boolean, whereas *rating* assumes ten categorical values of the form *'(0, 1]'*, *'(1, 2]'*, ..., *'(9, 10]'*. We can assume they represent the binning of a continuous rating, therefore we map them into an integer space [1,10] to facilitate later analysis. Just by looking at the head of the dataframe, we can also tell that *countryOfOrigin* and *genres* contain collections of categorical values; therefore, we transform them into lists for easier handling. Some discrete numerical attributes (*endYear*, *awardWins*) are stored as float rather than boolean because `int64` does not support NaN values; however, we do not deem it necessary to convert them.

A first global analysis of the dataframe's summary and descriptive statistics already gives us valuable insight into which variables have missing values[1], and into which features are uninformative: *bestRating*, *worstRating*, and *isRatable* can be safely dropped, as they only take one value for all records (10, 1, and 1, respectively). We can also see that *numVotes* and *ratingCount* share very similar, when not identical, statistics, so we drop *ratingCount* and only keep *numVotes*.

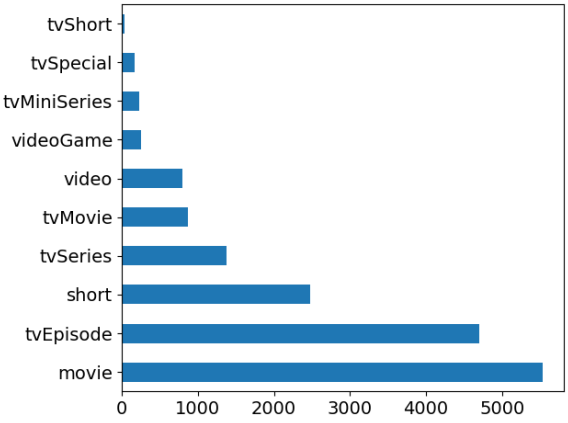

Figure 1: Barplot for *titleType*.

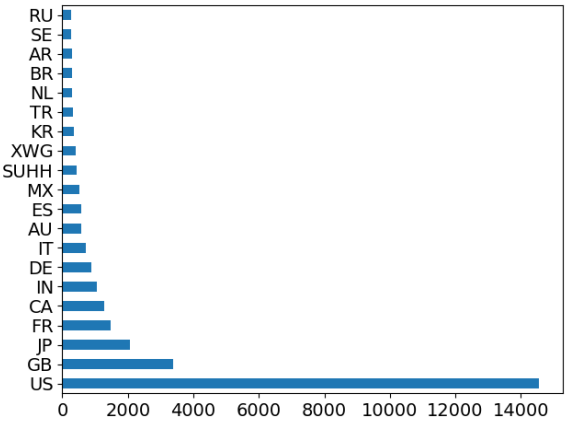

Figure 2: Barplot for *countryOfOrigin* (top 20).

1.2 Distribution of categorical variables and statistics

In an effort to analyze the univariate distribution of categorical variables, we produced barplots for *titleType* (Figure 1), *countryOfOrigin* (Figure 2), and *genres* (Figure 3). All distributions are imbalanced, being dominated by few very common classes: *movie* and *tvepisode*, the *US*, *drama* and *comedy*, respectively. A barplot for *rating*[2], in Figure 4, shows a left-skewed distribution, with the mode being the [7,8) category.

By looking at the categories for the different variables, we question the informativeness of *canHaveEpisodes* and *isAdult*. The first assumes positive values only for the title types *tvSeries* and

1 As the missing values were recorded inconsistently in the dataset, we used the `read_csv` function, passing a list of strings to recognize as missing values to the parameter `na_values`.

2 In this data exploration phase, we analyzed this ordinal variable both from a categorical and a numeric point of view, as to maximize insights.

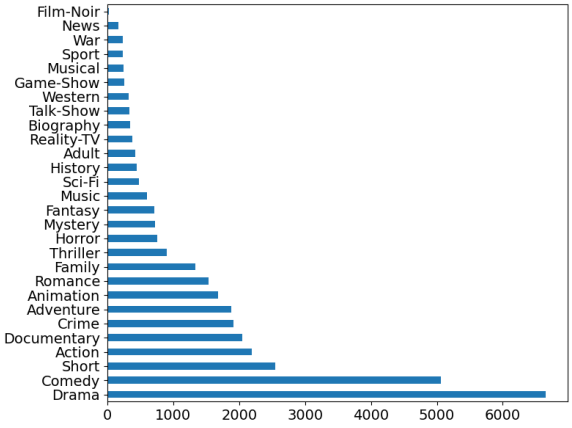

Figure 3: Barplot for *genres*.

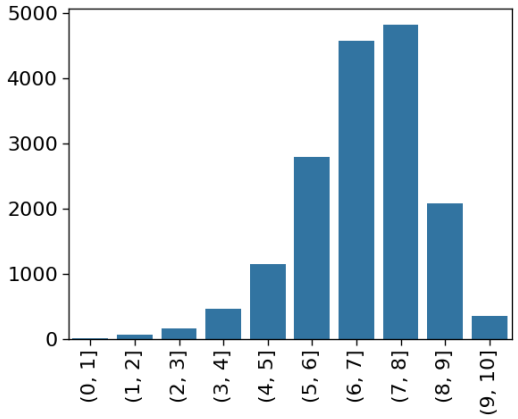

Figure 4: Barplot for *rating*.

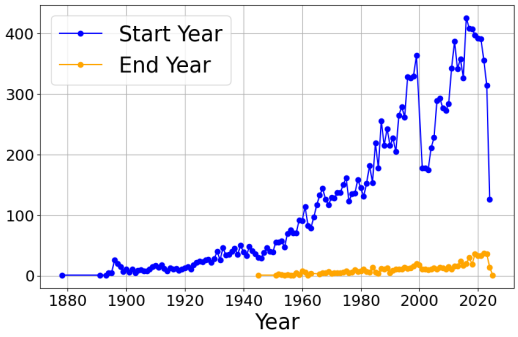

Figure 5: Time series: number of titles being released and ended by year.

tvMiniSeries: we keep it, as an explicit feature of serialized content. *isAdult* proves to be redundant as its information is conveyed by the *adult* category in *genres*, therefore we drop it.

1.3 Distribution of numeric variables and statistics; transformation of variables

Before analyzing numeric variables, we add the following features:

- *moreCountriesOfOrigin*: the number of countries listed for the record for the variable *countryOfOrigin*. For the training set, the values range from 1 to 10.
- *numGenres*: the number of genres listed in *genres*. For the training set, the values range from 1 to 3.
- *reviewsTotal*: the sum of *criticReviewsTotal* and *userReviewsTotal*.
- *criticReviewsRatio*: the proportion of review from critics (*criticsReviewTotal*) on the total number of reviews of a record (*reviewsTotal*). For unreviewed records, it is set to zero.
- *awardsAndNominations*: the sum of *awardWins* and *awardNominationsExcludeWins*.

Moreover, we use the available interval features, *startYear* and *endYear* to generate the timeline graph in Figure 5. We can observe how the number of titles being released each year has consistently grown until the end of the 2010s, with two noticeable dips in this trend: around 2001 (which could be linked to the aftermath of 9/11) and after 2020 (probably due to the Covid-19 pandemic). The time series for *endYear* follows the same trend on a smaller scale: we suspect a positive correlation between the two variables. There are no end dates available before the 1940s, which is understandable, as serialized content was not produced until around the 1930s.

In order to have a first global view of both the univariate distribution of the numeric variables singularly, and of possible interesting pairwise correlations, we build a pairplot with a KDE graph for each variable in the diagonal. Our previous intuition about a positive correlation between *startYear* and *endYear* is proved by the scatterplot in Figure 6, therefore we drop *endYear*, which is more problematic due to the large number of missing values.

The outliers visible in this plot highlight supposedly particularly long running television shows:

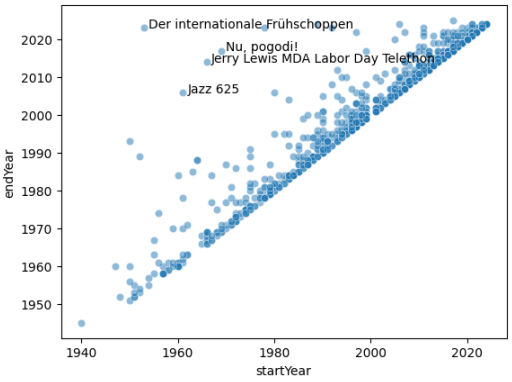

Figure 6: Scatterplot endYear~startYear.

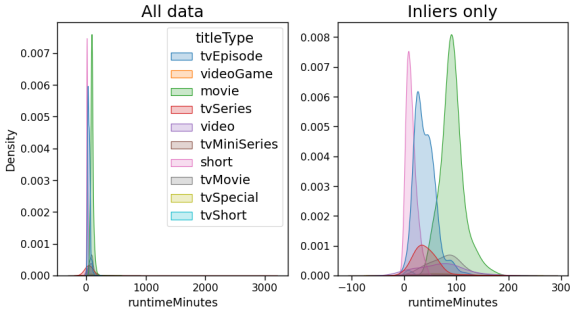

Figure 7: KDE for runtimeMinutes per title type, including and excluding outliers.

as a matter of fact, of those we analyzed, only the *MDA Labor Day Telethon* run continuously for 45 years, while the other titles refer to shows that were renewed or re-broadcasted after a long hiatus.

Most numeric features (*awardWins*, *numVotes*, *totalImages*, *totalVideos*, *totalCredits*, *criticReviewsTotal*, *awardNominationsExcludeWins*, *awardsAndNominations*, *numRegions*, *userReviewsTotal*, *numCountriesOfOrigin*, *reviewsTotal*) have heavily right-skewed distributions. It makes sense that the distribution of these types of features would approximate a power law. We can address this problem using a log transformation of these features.

On the other hand, *runtimeMinutes* also has a right-skewed distribution, but when excluding outliers (values higher than 3rd quartile + 1.5 IQR), we can see that, for the most part, runtimes are dependent on the *titleType* and that the distribution of the runtime for each title type approximates a bell curve (Figure 7); therefore, we prefer not to log transform this variable and be mindful of the presence of outliers. Examining the titles with a longer runtime reveals that part of the outliers is due to the inconsistent strategy used for the computation of the runtime for serialized titles (series and miniseries): in some cases, the episode runtime is provided, in others, it reports the runtime for the entire series, i.e. the sum of all its episodes' runtimes (some examples are reported in Table 1).

originalTitle	runtimeMinutes	titleType
Alim Day1	3000	tvSeries
Jerry Lewis MDA Labor Day Telethon	1290	tvSeries
Voice of the Planet	600	tvMiniSeries
Orbius	570	movie
Heritage: Civilization and the Jews	540	tvSeries

Table 1: Top 5 titles with the longest runtime.

1.4 Handling Missing Values

Four variables in the dataset have been found to contain missing values: *endYear* (15,617), *runtimeMinutes* (4,852), *awardWins* (2,618), *genres* (382).

In Subsection 1.3, we opted to drop *endYear* because its value was missing for most records. This decision is corroborated by the fact that, on one side, it heavily correlates with *startYear*, and on the other, the only records that do have an end year are of the type *tvSeries* and *tvMiniSeries*, and the information about the serial nature of media is already carried by the feature *canHaveEpisodes*.

awardWins has almost three thousand missing values. Given its highly unbalanced distribution, with the large majority of the records having no awards, it makes sense to fill them with the most frequent value, i.e. zero.

Analyzing the distribution of genres, we noticed that there is some variation in the frequency of different genres with respect to *titleType*: for example, while *Drama* is overall the most frequent genre (as can be seen in figure 3), *Comedy* tends to be more prevalent in serialized content. We decided to fill the 382 missing values for *genres* with the most frequent list of genres given the title type of the record (cf. Table 2).

As previously mentioned in Subsection 1.3, *runtimeMinutes* is dependent on the *titleType* of a record (cf. Figure 7): movies are generally longer than TV episodes, and shorts are typically shorter than both. Since we have some missing values for *runtimeMinutes*, we can use the median value for the respective type to fill them (cf. Table 2).

titleType	top genre	mdn runtime
movie	[Drama]	90
short	[Short]	12
tvEpisode	[Comedy]	40
tvMiniSeries	[Drama]	60
tvMovie	[Drama]	86
tvSeries	[Comedy]	31
tvShort	[Animation, Family, Short]	10
tvSpecial	[Comedy]	60
video	[Adult]	76
videoGame	[Action, Adventure, Fantasy]	28

Table 2: Top genre and median runtime (in minutes) for each *titleType*.

Value	awAndNoms	hasVids	moreCofOr
False	13692	14821	15285
True	2739	1610	1146

Table 3: Boolean value counts for *awardsAndNominations*, *hasVideos* and *moreCountriesOfOrigin*.

1.5 Pairwise correlations and eventual elimination of variables

We observed pairwise linear correlation among numerical variables (after operating the log transformation detailed in Subsection 1.3) by computing their correlation heatmap, displaying Pearson's correlation coefficient for each pair of features (Figure 8, left).

numVotes, *userReviewsTotal*, *criticReviewsTotal* and *reviewsTotal* are all highly positively correlated with each other (with a correlation coefficient greater than 70). We only keep *numVotes*, which has the least skewed distribution.

The majority of records have no awards nor nominations, no videos, and only one country of origin. Therefore we binarize *awardsAndNominations*, dropping both *awardWins* and *awardNominationsExcludeWins*, and we drop both *totalVideos* and *numCountriesOfOrigin*, substituting them for the boolean features *hasVideos* and *moreCountriesOfOrigin* (Table 3).

The overall number of features was reduced from 23 to 18; they are listed in Tables 6 and 7. Among the remaining numeric features, we observe in the heatmap in Figure 8 (right) that *numVotes* positively correlates to *totalImages* (0.61, moderate to high correlation), to *numRegions* (0.56, moderate correlation), and to *totalCredits* (0.51, low-moderate correlation).

2 CLUSTERING

2.1 Centroid-based Clustering

In analyzing the dataset using centroid-based methods, we experimented with two different algorithms, K-Means and Bisecting K-Means. For each experiment, we followed the same steps:

1. Choosing the attributes and scaling the dataset: we used all numeric features (cf. Table 6), log-transformed if necessary as detailed in Subsec. 1.3, and scaled the values using StandardScaler.

2. Selecting the best value for k: we used both the Elbow method and the Silhouette method as heuristics to determine the appropriate number of clusters. To do so, we iteratively run the algorithm for k in the range [2,50], plotting the SSE and Silhouette score for each k and comparing the two curves to find the best trade-off.

3. Fitting the algorithm and evaluating the resulting clustering in terms of cohesion and separation by calculating the SSE, Silhouette score, and correlation between the distance matrix of the dataset and the ideal similarity matrix.

4. Visual cluster exploration: cluster visualization through PCA, t-SNE, and a parallel coordinates plot of the centroids, followed by a further visualization of the clusters' composition w.r.t. categorical and numeric features.

2.1.1 *K-Means*

We select k=7 as a trade-off between SSE and Silhouette score. The resulting SSE and Silhouette score are 62588.082 and 0.189, respectively, while the correlation is -0.458. Though the SSE value might seem high, it is significantly lower than the SSE of 500 randomly generated datasets (cf. Figure 9), which suggests that our clustering is meaningful and not a product of chance.

The clusters' composition is reported in Table 8: in terms of population, we can distinguish Cluster 0 as the largest cluster, with almost 5000 records, followed by Cluster 2 with over 3000 records. The smallest cluster is Cluster 6, with only 950 records. The remaining clusters have between 1400 and 2000 records.

The first thing that emerges from the parallel coordinates graph (Figure 10) is how Cluster 3 is well-separated from the others w.r.t half of

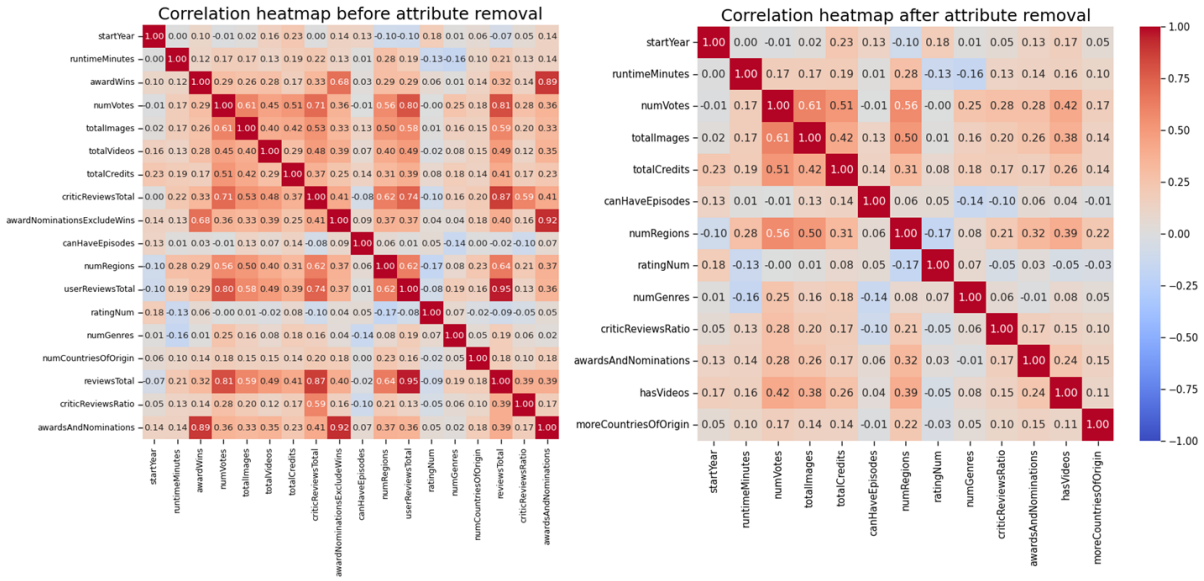

Figure 8: Correlation heatmaps, before and after attribute removal.

Feature	Description	Type	dtype
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.	float64
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.	int64
endYear	TV Series end year.	Discrete. Interval.	float64
ratingCount	The total number of user ratings submitted for the title.	Discrete. Ratio-scaled.	int64
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled.	int64
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled.	int64
totalImages	Total Number of Images for the title within the IMDb title page.	Discrete. Ratio-scaled.	int64
totalVideos	Total Number of Videos for the title within the IMDb title page.	Discrete. Ratio-scaled.	int64
totalCredits	Total Number of Credits for the title.	Discrete. Ratio-scaled.	int64
criticsReviewTotal	Total Number of Critic Reviews.	Discrete. Ratio-scaled.	int64
awardWins	Number of awards the title won.	Discrete. Ratio-scaled.	float64
awardNominationExclWins	Number of award nominations excluding wins.	Discrete. Ratio-scaled.	int64
userReviewsTotal	Total Number of Users Reviews.	Discrete. Ratio-scaled.	int64

Table 4: Numeric attributes.

Feature	Description	Type	dtype
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e., names).	object
titleType	The type/format of the title (e.g., movie, short, episode, video, etc.).	Categorical, 10 classes.	object
countryOfOrigin	The country where the title was primarily produced. Some titles can belong to more than one class.	Categorical, 153 classes. A record can belong to more than one class.	object
genres	The genre(s) associated with the title (e.g., drama, comedy, action). Some titles can belong to more than one class.	Categorical, 29 classes. A record can belong to more than one class.	object
canHaveEpisodes	Whether or not the title can have episodes.	Asymmetric binary.	bool
isRatable	Whether or not the title can be rated by users.	Asymmetric binary.	bool
isAdult	Whether or not the title is for adults.	Asymmetric binary.	int64
rating	IMDB title rating class.	Ordinal, 10 values.	object
worstRating	Worst title rating.	Ordinal, supposedly [1,10].	int64
bestRating	Best title rating.	Ordinal, supposedly [1,10].	int64

Table 5: Categorical, ordinal, and binary attributes.

Feature	Description	Type
runtimeMinutes	Primary runtime of the title, in minutes.	Continuous. Ratio-scaled.
startYear	Represents the release year of a title. In the case of TV Series, it is the series' start year.	Discrete. Interval.
numVotes	Number of votes the title has received.	Discrete. Ratio-scaled. Log-transformed.
numRegions	The regions number for this version of the title.	Discrete. Ratio-scaled. Log-transformed.
totalImages	Total number of images for the title within the IMDb title page.	Discrete. Ratio-scaled. Log-transformed.
totalCredits	Total number of credits for the title.	Discrete. Ratio-scaled. Log-transformed.
criticReviewsRatio	The proportion of reviews from critics on the total number of reviews of a record. For unreviewed records, it is set to 0.	Continuous. Ratio-scaled.
awardsAndNominations	Total number of awards and nominations.	Discrete. Ratio-scaled. Log-transformed.
numGenres	The number of genres listed for the variable *genres*.	Discrete. Ratio-scaled.

Table 6: Numeric attributes after data preparation.

Feature	Description	Type
originalTitle	Original title, in the original language.	Categorical; mostly unique classes (i.e. names).
titleType	The type/format of the title (e.g. movie, short, tvseries, tvepisode, video, etc.).	Categorical, 10 classes.
countryOfOrigin	The country where the title was primarily produced.	Categorical, 153 classes. Some titles can belong to more than 1 class.
genres	The genre(s) associated with the title (e.g., drama, comedy, action).	Categorical, 29 classes. Some titles can belong to more than 1 class.
rating	IMDB title rating class.	Ordinal, 10 values.
ratingNum	Mapping of *rating* to the integers [1,10].	Ordinal, 10 values.
canHaveEpisodes	Whether or not the title can have episodes.	Asymmetric binary.
hasVideos	Whether or not the title IMDb title page has at least one video.	Asymmetric binary.
moreCountriesOfOrigin	Whether or not the title was produced in more than one country.	Binary.

Table 7: Categorical, ordinal, and binary attributes after data preparation.

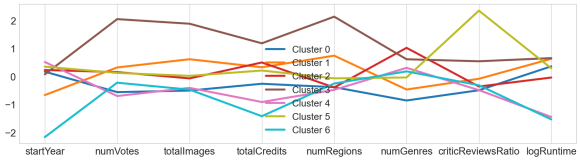

Figure 10: Centroids' visualization for K-Means.

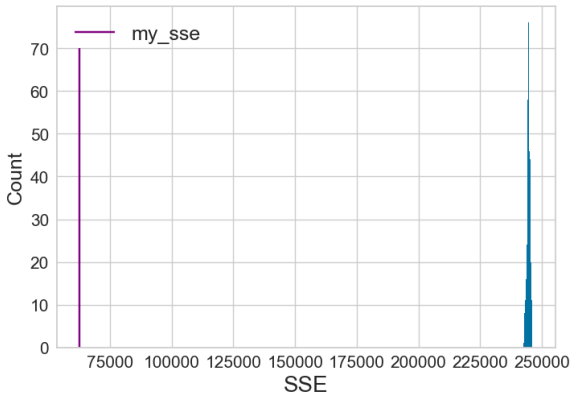

Figure 9: Statistical framework for clusters' validity: SSE (K-Means).

the features: it has a particularly high value for *numVotes*, *totalImages*, *totalCredits*, and *numRegions* – variables for which we had observed moderate correlation in Subsec. 1.5 . Cluster 5 has a significantly higher value for *criticReviewsRatio*: the algorithm might have isolated titles with greater critical appeal. Clusters 6 and 4 have the lowest runtime by far; Cluster 6, the smallest one, is also characterized by a low value for *startYear* and *totalCredits*: it probably contains the oldest and shortest titles. Clusters 0 and 2 appear to be the least-well separated from the others; they are indeed the biggest clusters.

In order to evaluate the results, we first used stratified bar charts to visualize the distribution of the clusters against known categorical features.

- *titleType* (cf. the heatmap in Figure 11): the smallest cluster, Cluster 6, is the purest, being mainly made up of shorts. Nevertheless, it's Cluster 4 that contains the largest num-

Clust.	# of records
0	4841
1	1985
2	3424
3	1424
4	1985
5	1822
6	950

Table 8: Clusters for K-Means.

Clust.	# of records
0	1881
1	1398
2	1397
3	1712
4	3595
5	4184
6	2264

Table 9: Clusters for BK-Means.

ber of shorts (as well as some tvEpisodes). It also stands out for having the fewest movies —— a class otherwise well represented across all other clusters. Cluster 2 has a very large proportion of tvEpisode, which explains its centroid's medium-low position for runtime in the parallel coordinates plot. The importance of *logRuntime* for explaining this clustering is also highlighted by the fact that the few tvShorts present have been almost entirely distributed amongst these three clusters. Cluster 0, the biggest, contains a great number of almost all title types.

- Boolean variables: for almost all clusters, the True records represent a small proportion, except for Cluster 3: around half of its records are True for *hasVideos* and *awardsAndNominations*. Given the insights from the parallel coordinates plot, we can assume this cluster contains mainstream, popular content: it has relatively more videos, images, and votes on its page and was distributed in more countries, but it doesn't have the highest ratio of critical reviews. Cluster 6, on the other hand, has little to no True values for these variables. The highest proportion of True values for *canHaveEpisodes* is in Cluster 0, which contains the largest amount of tvSeries and tvMiniSeries.

We also analyzed the clusters' composition w.r.t. the numeric variables used by the algorithm, expanding the insights gained from the parallel coordinates plot. Interestingly, the two purest clusters w.r.t to *numGenres* are the biggest: Cluster 0 mostly has one genre, with a minority of two genres, while Cluster 2 mostly has three genres, with a small proportion of two genres. The other clusters are more heterogeneous, having all three possible values. The line graph in Figure 12 shows the distribution of the clusters w.r.t. *startYear*: as hinted by its centroid, Cluster 6 contains the majority of records released before 1930s, and has no record released after 1983. On the other hand,

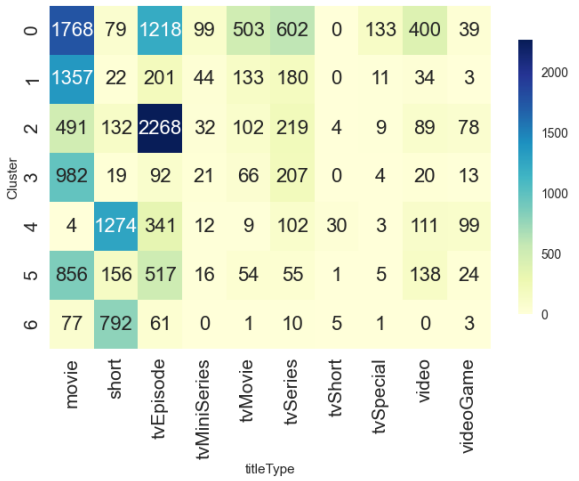

Figure 11: Heatmap Clusters vs titleType for K-Means.

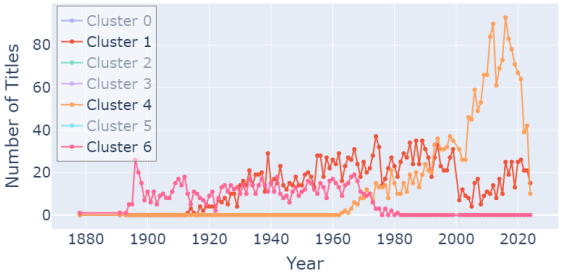

Figure 12: Number of titles by year for relevant clusters for K-Means.

Cluster 4 only has records released since 1963: the algorithm has therefore grouped older shorts in a cluster and newer shorts in another. Cluster 1, which also had a relatively low centroid, mostly contains records from the 1930s until the 1990s. Boxplots for the remaining variables also confirm the insights that were previously discussed. For *runtimeMinutes*, Clusters 4 and 6 have the lowest median, at 12 minutes, as well as a short box, indicating low variability. Cluster 4 follows at 40 minutes, coherently with the lenght of a tvEpisode.

2.1.2 *Bisecting K-Means*

We select k=7, the only significant local peak in the Silhouette plot (Figure 13). All evaluation metrics are worse than the ones for K-Means, with a SSE equal to 67857.023, a Silhouette score equal to 0.128, and a correlation of -0.393. Again, we compared the SSE with the one of 500 randomly generated datasets to confirm its meaningfulness. The clusters' sizes are more homogeous than K-Means' (cf. Table 9): Cluster 5 is the biggest with around 4000 records, followed by Cluster 4 with

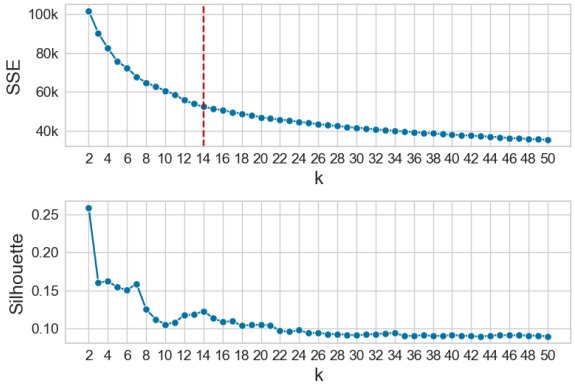

Figure 13: SSE and Silhouette score for choosing k (BK-Means).

Figure 14: Centroids' visualization for BK-Means.

almost 3600 records. All other clusters range between around 1400 and 2300 records.

Analyzing the parallel coordinates plot in Figure 14, we can see that similarly to before, there's a cluster, Cluster 2, that is well-separated from the others w.r.t. *numvotes*, *totalImages*, *totalCredits* and *numRegions*. In terms of these values, it is followed by Clusters 0 and 1. Moreover, there's once again a cluster, Cluster 1, with a particularly high value for *criticReviewsRatio*. Differently from K-Means, there is one single cluster, Cluster 3, with a significantly lower value for both *logRuntime* and *totalCredits*. It also has the lowest value for *startYear*, together with Cluster 6. Differently from before, it seems that the algorithm has created a single cluster for shorts, that probably contains also newer titles. Cluster 4 also has a relatively low value for *logRuntime*; we expect it to contain tvEpisodes and the remaining shorts.

Like for K-Means, we visualized the distribution of the clusters against known categorical features. We find many similarities with the previous algorithm, but also some differences:

- *titleType* (cf. Figure 15): again, the distribution of shorts is what becomes immediately evident. Around half of them are to be found in Cluster 3, which also has a small proportion of tvEpisodes and is the only cluster with almost no movies. Cluster 4, the 2nd biggest cluster, is mostly made of tvEpisodes and a smaller, yet significant, proportion of shorts.

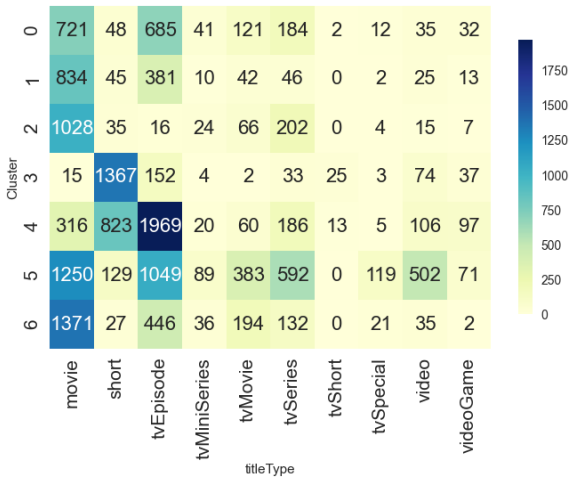

Figure 15: Heatmap Clusters vs titleType (BK-Means).

The few tvShort recorded are also distributed among these two clusters. It's also worth noting that Cluster 6 contains the biggest number of movies, although being only the 3rd most popular, and that Cluster 2 is the only cluster with almost no tvEpisodes. Cluster 5, the biggest, contains the largest part of the remaning title types (except for videogame, which are mostly in Cluster 4).

- Boolean variables: again, the cluster whose centroid had the highest value for *numvotes*, *totalImages*, *totalCredits* and *numRegions*, Cluster 2, also has the highest proportion of True values for *hasVideos* and *awardsAndNominations*, at around 50%. The biggest cluster has the most True values for *canHaveEpisodes* because it contains most of the series. Cluster 6 has almost no True values for *hasVideos*, probably because it contains older titles.

By visualizing the clusters' composition w.r.t. the numeric variables used by the algorithm, we can see how the line graph for *startYear* (Figure 16) shows important differences to the one for K-Means. Cluster 3 (the shorts' cluster) does contain the majority of records released before 1930s, but it actually has titles from all years: we were correct in assuming that it contains both older and newer shorts. Cluster 6 only has records released between the 1910s until 1999, and contains the majority of records released from 1955 until 1987. Cluster 5 (the biggest one) only contains newer titles, released since the 1980s. W.r.t. *numGenres*, only Cluster 4 has no records with one genre: it contains the largest amount of records with three genres, as well as some with two. Clusters 5 and 6 are also quite pure, with a majority of records

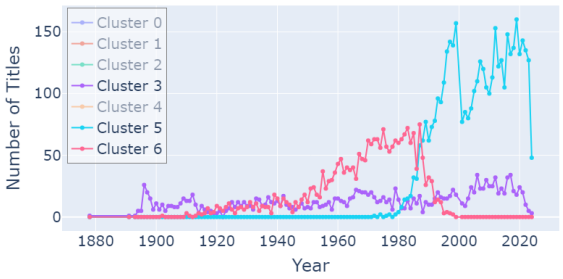

Figure 16: Number of titles by year for relevant clusters for BK-Means.

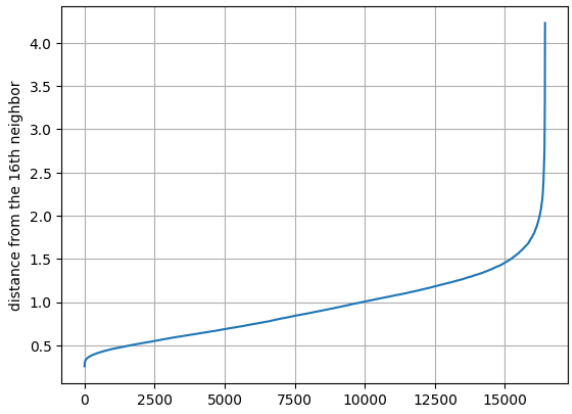

Figure 17: Sorted distances from the 16^{th} neighbor.

with one genre, a minority with two, and a negligible amount with three.

In conclusion, although Bisecting K-Means has yielded worse metrics than K-Means, it has still produced meaningful clusters, comparable to those from K-Means. Both algorithms have created one or two clusters containing shorts, one containing a large portion of tvEpisodes, a cluster containing mainstream/popular content, and one with critically interesting titles. Neither clustering is separated w.r.t the rating (for both, the clusters are normally distributed against *ratingNum*).

2.2 Density-based clustering

2.2.1 DBSCAN

In order to choose the hyperparameters ϵ and *MinPts*, we experimented with plotted the sorted distances from the k^{th} neighbor for several values of k (powers of 2 between 8 and 512) and checked for the presence of an elbow in the plot. We then fixed *MinPts* to a value of k (we choose 16, since all values of k gave use very similar curves) and used the elbow in the plot to choose a suitable value for ϵ.

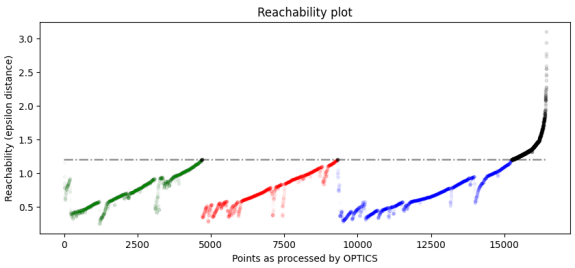

Figure 18: Reachability plot produced by OPTICS with *MinPts*=16

In fig. 17 we can see that the elbow of the curve is around 1.5, so we run the clustering algorithm with *MinPts*=16 and ϵ=1.5. DBSCAN classified almost 99% of our titles as noise points and clustered together all remaining titles.

2.2.2 OPTICS

Since we failed to detect significative clusters in the data determining ϵ with the elbow method, we decided to run OPTICS with the same value of *MinPts* and inspect the reachability plot.

In fig. 18 we can see that the reachability plot doesn't show any real valleys which would allow us to detect significant clusters. It is possible that our data is simply too noisy to be suitable for this type of analysis. Still, setting ϵ to 1.2, we were able to extract 3 clusters of similar sizes and remove 1188 noise points. This clustering obtained a Silhouette score of 0.09. The only notable trend found in the analysis of this clustering is that they cleanly split the titles by number of genres.

2.3 Hierarchical Clustering

In analyzing the dataset using agglomerative hierarchical clustering we initially entertained the idea of using a precomputed distance matrix in order to include categorical attributes. However we ultimately decided against this and used only the numeric features with euclidean distance in order to obtain results that are comparable with those obtained with centroid based methods, be able to use Ward's method for determining inter-cluster distances and escape the curse of dimensionality. This also enabled us to check if clusters computed on the basis of numerical features were able to capture information vehicolated by categorical features.

We performed clustering using all linkage criteria made available from the scikit learn library (single linkage, average linkage, complete linkage and Ward's method), visualized the dendrogram

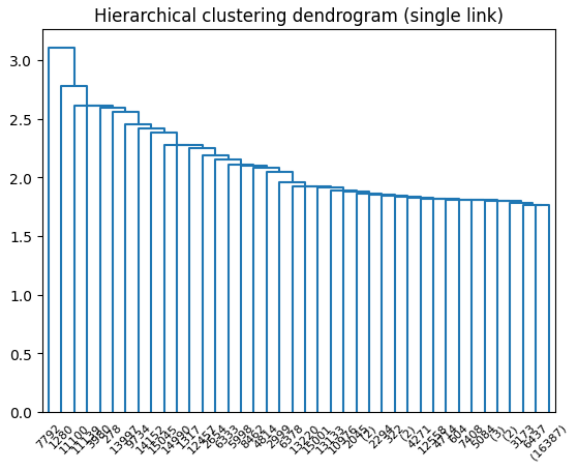

Figure 19: Dendrogram for hierarchical clustering obtained with single linkage.

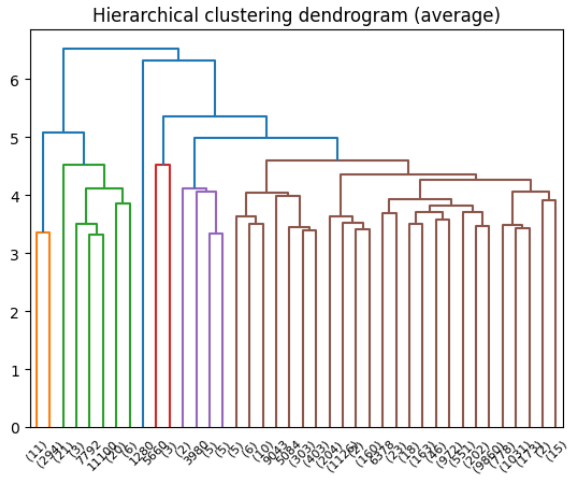

Figure 20: Dendrogram for hierarchical clustering obtained with average linkage (threshold 4.7.)

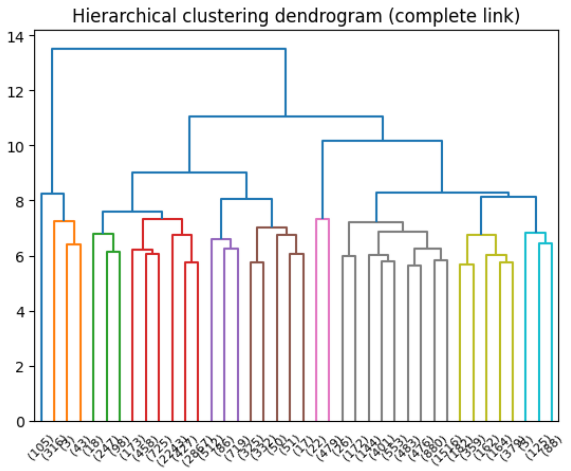

Figure 21: Dendrogram for hierarchical clustering obtained with complete linkage (threshold 7.5).

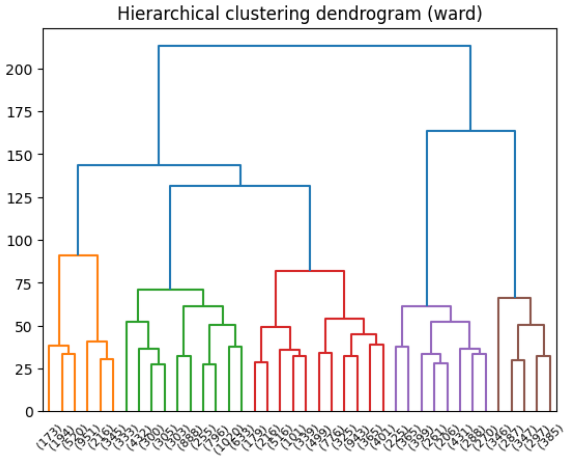

Figure 22: Dendrogram for hierarchical clustering obtained with Ward method (threshold 100).

(figg. 19-22) for each clustering and selected an appropriate threshold value for further analysis.

The **single linkage** criterion failed to yield an interesting clustering, as can be seen from the dendrogram at figure 19. Cutting this dendrogram at any height would create several singleton clusters. This is an expected result, since single linkage performs poorly with noisy data.

Group average performed slightly better (fig 20), but produced a very unbalanced clustering: setting the threshold at 4.7 we obtained 5 clusters, with one single cluster (Cluster 5) containing 98% of the records, as can be seen in fig 23. We obtained a Silhouette score of 0.206.

Complete linkage also demonstrated the *rich get richer* tendency of agglomerative clustering, producing clusters of variable size. We cut the dendrogram (fig. 21) at 7.5, obtaining 10 clusters with two major cluster containing more than 70% of all records and several smaller clusters, six of which having less than 1,000 members. We obtained a Silhouette score of 0.056.

Ward's method obtained the most balanced clustering. We cut the dendrogramm (fig. 22) at 100, obtaining 5 clusters and a Silhouette score of 0.157.

We then evaluated the clusterings obtained with complete linkage and Ward's method with respect to the distribution of of categorical features.

We found out that both Ward and complete linkage were able to create clusters containing mainly shorts: Ward's Cluster 1 captures 76% of shorts and 85% of tv shorts of the whole dataset and 80% of it members are shorts or tv shorts; for complete linkage, Clusters 5 and 6 capture, in between them, 69% of the total shorts and 87% of tv shorts and are themselves fairly pure with

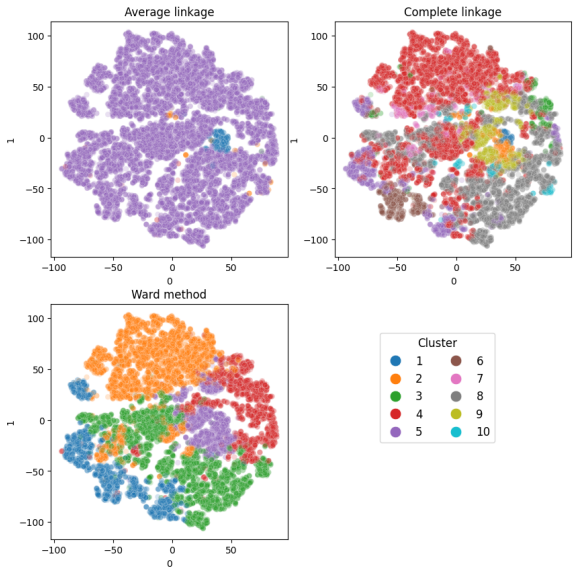

Figure 23: Clusters obtained with different criteria. Visualized by tSNE.

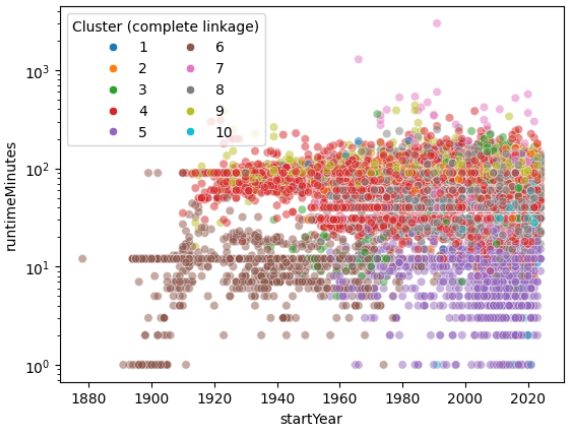

Figure 24: Complete linkage clustering by year of release and runtime.

respect to title type (78% of members of Cluster 5 and 94% of members of Cluster 6 are shorts or tv shorts). Figure 23 shows notable overlap between the shorts clusters obtained by these two methods.

In fig 24 we show the results of clustering with complete linkage by year of release and runtime. Here we can see that Cluster 4 (which is the most numerous), containing mainly movies and tv episodes is cleanly separated from the two shorts clusters by means of runtime. We can also see that the two clusters of shorts detected by the algorithm differ for date of release: Cluster 6 contains older shorts, while Cluster 5 contains shorts that were realeased from the 60s onwards. From the dendrogram in fig. 21, we can see that, had we selected a higher threshold for splitting the dendrogram, these two clusters would be merged together.

Some of the smallest clusters obtained with complete linkage also proved interesting. In fig. 25 we can see that Cluster 1 and 2 (that contain mainly movies and some tv series and are merged at a higher threshold) contain titles that are on average more popular that all other titles in the dataset (they have a higher number of votes), but the ones in Cluster 1 are the most popular: this cluster contains blockbusters like *Harry Potter and the Deathly Hallows – Part 2*, *Il buono, il brutto, il cattivo* and *Full metal jacket*, just to name a few, as well as very popular TV series like *The Witcher* and *Euphoria*.

Moreover, almost all (96%) records from cluster 1 received awards or nominations and almost all of them (94%) have attached videos on their IMDB page. Of the slightly less popular records from Cluster 2, 65% received awards or nominations, while 72% have videos on their IMDB page, while the values for the entire database are much lower (respectively 17% and 10%).

Similar results emerge in the clustering with average linkage, where 83% of the 305 records of Cluster 1 (which in figure 23 is shown to overlap significantly with the previous two clusters) received awards and nominations and 91% have attached videos on their IMDB page.

It should be mentioned that the aforementioned clusters capture only a small part of all records that received awards or nominations, but this is, nevertheless, an interesting result, since we only used the numerical features for the clustering, so no information about awards or videos was provided to the model.

3 CLASSIFICATION

In this section we examine the performance of various classification methods with respect to two multiclass classification tasks.

Since we noticed promising trends in the cluster analysis, we chose *titleType* as target variable for the first task.

In the second task we attempted to predict the rating of a title. Since we wanted to avoid having a great number of classes with very low support, we binned the rating values in four categories, each containing approximately the same number of records, as follows:

LOW rating rating lower than 6

MEDIUM-LOW rating between 6 and 7

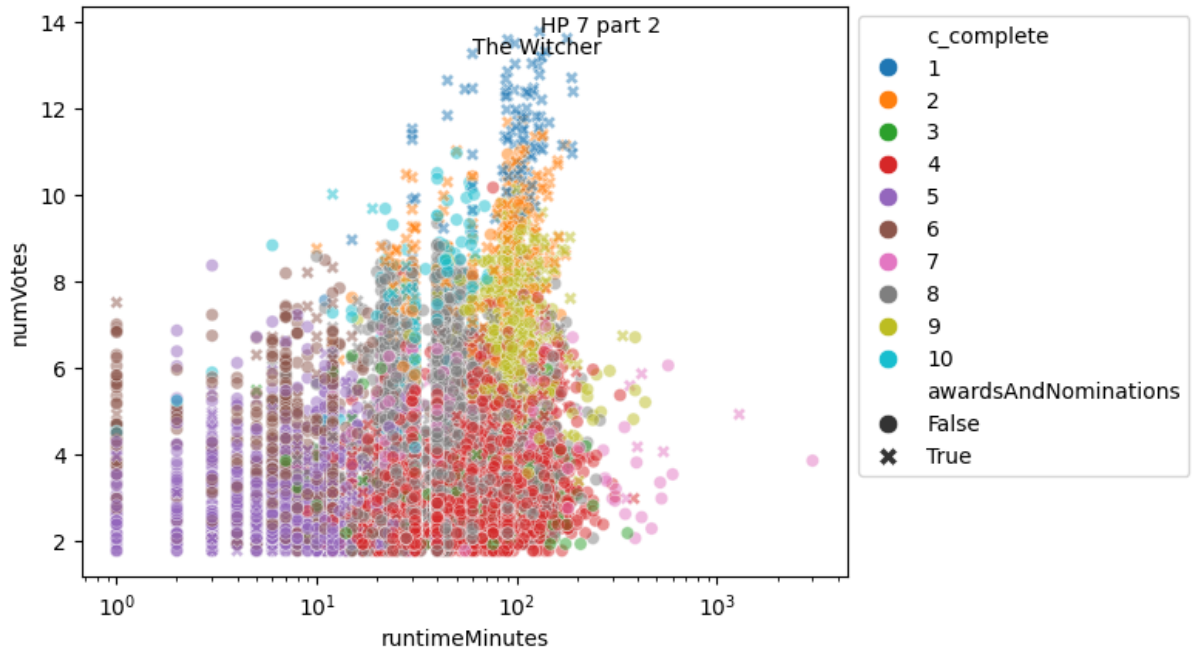

Figure 25: Complete linkage clustering by runtime and number of votes

MEDIUM-HIGH rating between 7 and 8

HIGH rating 8 or higher

For this second task we expected overall worse results, because none of our features showed high correlation with rating and we also saw that our records didn't cluster by rating in any significant way.

In evaluating our models, we considered not only the accuracy score, but also the macro average of F1 scores obtained for each class. Since the classes for the first have variable support, we reasoned that this second metric would be a better indicator of the overall performance of the models.

3.1 K-Nearest Neighbor Classifier

For each target feature we trained the knn classifier on all numerical features (log transformed if necessary) which were normalized using a standard scaler.

In both cases we performed grid search in order to find the optimal configuration of hyperparamenters: we tested different values of k between 2 and 500, two different distance functions (manhattan and euclidean) and wheter to weight exemplars by distance. We evaluated the performance of each configuration of hyperparameters performing a 5-fold stratified cross validation on the training set and checking the mean accuracy. In order to limit the time used on this task, we performed the search in two steps, as illustrated by figg. 26 and 27:

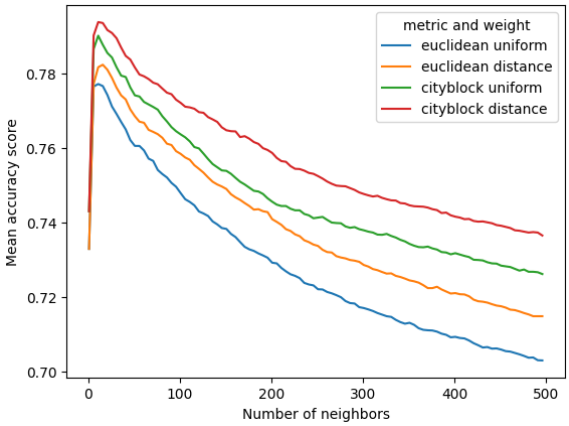

Figure 26: Accuracy of knn models evaluated during grid search (target: titleType).

1. We performed grid search with values of k between 2 and 500 using a step of 5;
2. After visually inspecting the trend of the mean accuracy on our first grid search by means of a lineplot, we refined our search for k to a restricted range, this time using a step of 1.

3.1.1 *Target:* titleType

We fitted and tested the classifier with the following configuration of parameters:

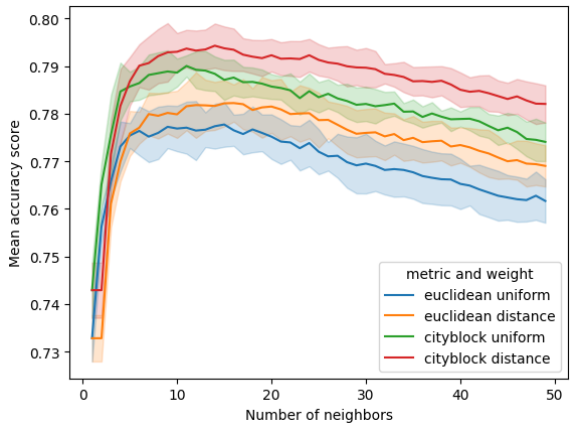

Figure 27: The search is refined to a limited range of k (target: titleType).

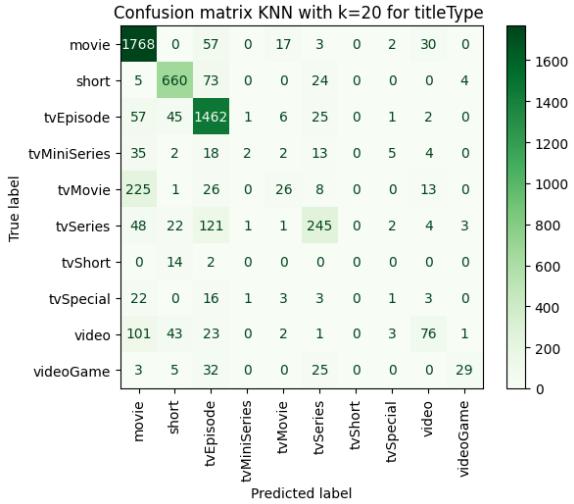

Figure 28: Confusion matrix for KNN for *titleType*

- distance function: Manhattan
- weight: distance
- k: 20 (as can be seen in fig. 27 *k*=14 obtained the highest mean accuracy score, but we preferred 20 because of the lower standard deviation).

This model obtained an accuracy of 0.78 and a macro F1 score of 0.42 on the test set, well exceeding the performance of the dummy classifier.

In particular, as can be seen from the confusion matrix in figure 31, the model has little troble identifying the three classes with higher support (movies, shorts and TV episodes have all F1 scores higher than 0.84) and is able to correctly detect tvSeries around half of the time. We can also see that tv movies, miniseries and videos are most frequently misclassified as movies, while tv shorts are almost always classified as shorts. TV specials are sometimes misclassified as tv episodes (especially if shorter) and sometimes as movies (especially if longer).

3.1.2 *Target:* binned rating

We fitted and tested the classifier with the following configuration of parameters:

- distance function Manthattan
- weight: distance
- k: 95

This model obtained an accuracy of .44 and a macro F1 score of .34, exceeding the performance of the baseline in both metrics. Overall the performance on this target variable was less satifactory, as expected.

The inspection of the confusion matrix revealed confusion among all classes (not only adiacent classes), with only *low* and *medium high* obtaining a F1 score greater than .50. The worst performing class was *high*, with an especially bad recall (0.14): more than half of the time titles with a high score were misclassified as *medium high*, with the rest of the cases distributed seemingly at random in the remaining classes.

3.2 Naïve Bayes Classifier

For each target variable, we build two models, one using only numerical features (Gaussian NB), and one using all features except the target variable (Categorical NB). The latter requires preprocessing: we discretize the numerical features into quartile-based bins and encode categorical features using one-hot encoding. For *countriesOfOrigin* we only keep the 40 most frequent countries to reduce dimensionality. Given their nature, Naïve Bayes Classifiers don't require feature scaling or feature selection: they rely on probablities rather than distances and are robust to irrelevant features.

3.2.1 *Target: titleType*

Both models outperform the baseline, but the CategoricalNB model performs significantly better than the GaussianNB model (cf. Table 10).

Model	Accuracy	Macro F1	ROC AUC	PR AUC
GNB	0.676	0.398	0.880	0.442
CNB	0.773	0.557	0.944	0.584

Table 10: Performance of GaussianNB and CategoricalNB models on *titleType*.

Analysing the **GaussianNB** model, we can immediately see that the classes with a higher support have better performances, with the most popular class, movie, scoring >0.80 in all metrics. As expected, the class it is mostly mistaken for, with 98 misclassificated records, is tvMovie. The contrary is also true, with the relatively small tvMovie class being classified as movie most of the time. The 2nd most popular class, tvEpisode, has a lower precision, at 0.70: its variable runtime leads to some misclassifications (mostly as movie, short or tvSeries). The 3rd most popular class, short, performs almost as well as movie while having less than half its support (P: 0.79, R: 0.78): as we've seen in clustering, this class separates itself from the others particularly well. As for the remaining underrepresented classes, they get often misclassified as more popular classes with which they share some similarities: tvShort is almost always classified as short (for its runtime); tvSpecial, tvSeries and tvMiniSeries have largely been classified as tvEpisode; videogame has a good precision (0.86) but was also mistaken for tvEpisode a significant amount of times; video, being a broad classification for a nonuniform set of titles, has been misclassified as different classes.

Moving onto the **CategoricalNB** model, the performance for almost all classes improves, with all three most popular classes now scoring >0.80 in all metrics. tvMiniSeries and tvSeries show the most dramatic improvement, with the latter scoring the highest precision (0.921) and F1 (0.904). They are now both correctly classified most of the time, and the model has managed to distinguish them from tvEpisode: they now only get misclassified as each other (42% of the time for tvMiniSeries, only 9% for tvSeries). The two PR curves in Figure 29 and 30 show this improvement, as well as the decrease in performance for videogame, which has gained recall at the cost of precision.

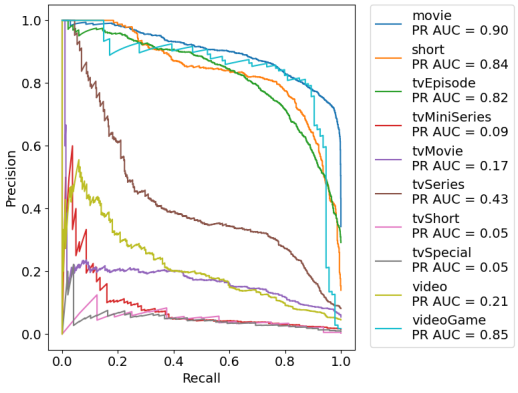

Figure 29: PR curves (1-vs-rest) for GaussianNB.

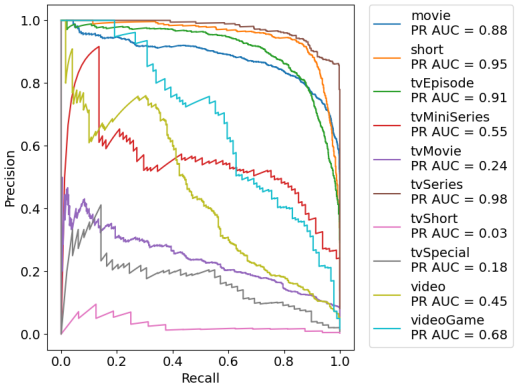

Figure 30: PR curves (1-vs-rest) for CategoricalNB.

3.2.2 *Target: binned rating*

On this task, the performance of the two models was less satisfactory. Nevertheless, both outperformed the baseline, with the CategoricalNB model performing once again better than the GaussianNB model, though less markedly (cf. Table 11).

Model	Accuracy	Macro F1	ROC AUC	PR AUC
GNB	0.345	0.330	0.630	0.348
CNB	0.427	0.387	0.680	0.398

Table 11: Performance of GaussianNB and CategoricalNB models on *binned rating*.

For the GaussianNB model, the confusion matrix (Figure 31) shows confusion among all classes, in particular w.r.t Medium-High, the most popular class (which is the one with the best recall and F1, 0.494 and 0.428): it gets predicted the most. Medium-Low has a particularly low recall (0.173, the benchmark is 0.275): its records are classified more frequently as any other class (especially as Medium-High and Low) than as itself. High and Medium-High are mostly confused among themselves, this is especially true for High, which has the lowest precision (0.251).

The Categorical NB model's overall performance is better than the Gaussian's, but actually this is mostly due to the great improvement of the Low class (F1 went from 0.361 to 0.55): 2/3 of its records are now classified correctly, the misclassified ones are quite balanced among the three remaining classes. All other classes have has a slight decrease in macro F1. Medium-Low has had a decrease in recall for a slight improvement in precision; less records are classified correctly, but now it is mostly confused with Low. Medium-High and High both had an improvement in precision, at the cost of recall. The improvement in this model's scoring and ranking capability is better

visualized by the PR curves in Figure 32: the precision for Low and Medium-High is more stable across different thresholds.

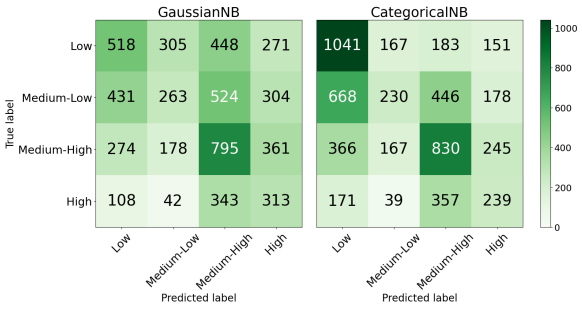

Figure 31: Confusion matrixes for GaussianNB and CategoricalNB for *binned rating*.

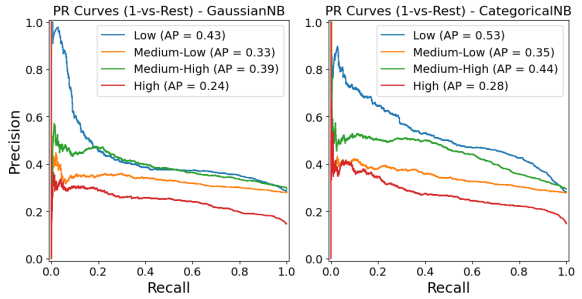

Figure 32: PR curves for GaussianNB and CategoricalNB for *binned rating*.

3.3 Decision Tree Classifier

We build two models for each target variable, one using only numerical features and one using all features except the target variable. For categorical features we repeat the preprocessing detailed in Subsec. 3.2. Decision Trees (DTs) do not require feature scaling, but they are prone to overfitting, therefore we apply pre-pruning and post-pruning techniques. We use a random search to find the best combination of hyperparameters for each model from the following grid:

```
{'max_depth': [None] + list(np.arange(2, 20)),
'min_samples_split': [2, 5, 10, 15, 20, 30, 50, 100],
'min_samples_leaf': [1, 5, 10, 15, 20, 30, 50, 100],
'criterion': ['gini', 'entropy']}
```

The random search is performed with 5-fold stratified cross-validation, repeated 5 times. Keeping the best hyperparameters found fixed, we then find the value of the *ccp_alpha* parameter that maximises the average macro F1 score of the validation sets in a 5-fold stratified cross-validation as a form of post-pruning.

3.3.1 *Target: titleType*

Both models not only outperform the baseline, but also both Naïve Bayes models. The DT using all features has better performances, especially in terms of macro F1 and PR AUC (cf. Table 13): it is superior both at fixed thresholds and across the score distribution.

Model	max_ depth	min_s _split	min_s _leaf	criterion	ccp_ alpha
Num.	10	50	10	gini	2.67e-4
All.	13	30	5	entropy	3.37e-4

Table 12: Best hyperparameters for DTs on *titleType*.

Model	Accuracy	Macro F1	ROC AUC	PR AUC
Num.	0.827	0.578	0.931	0.585
All	0.846	0.661	0.913	0.682

Table 13: Performance of DTs models using only numeric vs all features on *titleType*.

Analysing the classification report and confusion matrix of the **numeric-only DT**, we can see that the three most popular classes have the best scores (macro f1 >= 0.845), especially in terms of recall. Videogame, although having a lower support, performs similarly (macro f1 = 0.832), being barely misclassified. The **second model** shows an increase in performances, especially for the less popular classes, both in terms of PR curve and macro F1. The models has learnt to distinguish tvMiniSeries and tvSeries from the rest, with the latter having the 2nd highest macro F1 (0.940). The best performing class is now videogame (macro F1 = 0.945), which is classified correctly most of the time. Looking at their **feature importances**, both models rely on *logRuntime*, but while the first one does so heavily (0.756), the second one less so (0.451). Unsurprisingly, the 2nd most important feature for the 2nd DT is *canHaveEpisodes*, followed by the genre *Short*.

In fact, for both trees the first split is on *logRuntime*<= 4.119 (≈ 60.5 minutes): this already allows to separate all shorts, tvShort, and videogames. For the **numeric DT**, the second split on *logRuntime*<= 3.068 (≈20.5 minutes) separates medium-lenght tvMovies and movies of the left subtree, as well as all videogames (minus 1). On the right subtree, splitting the longer titles based on having more than an image separates among them most of the movies, tvEpisodes and tvMovies. For the **all-features DT** (Figure 33), in two more splits we get to a node in the right subtree wich will lead to a leaf: it's adult videos that are longer than an hour and released since 1991. On the left subtree,

we get to a leaf contaning *tvSeries*in 3 total splits as well: the are at most an hour long, have *Short* as a genre, and can have episodes. It's also worth noting that all videogames (minus 1) are again isolated in just two splits: they are shorter than 61 minutes and are tagged as genre *Short*.

3.3.2 Target: binned rating

In this case, each model overall outperforms the baseline and its corresponding Naïve Bayes model; again, the all-features DT is the best performing one (cf. Table 15).

Model	max_ depth	min_s _split	min_s _leaf	criterion	ccp_ alpha
Num.	6	2	50	gini	0.0
All.	11	100	5	entropy	4.49e-4

Table 14: Best hyperparameters on *binned rating*.

Model	Accuracy	Macro F1	ROC AUC	PR AUC
Num.	0.419	0.341	0.678	0.388
All	0.443	0.405	0.704	0.429

Table 15: Performance of DTs models using only numeric vs all features on *binned rating*.

The most notable thing for the **numeric-only DT** is the increadibly low recall for class High (0.06), lower than the dummy classifier's (0.135): this class only had 2417 records in the training set, while all other had over 4500 – the model hasn't generalized on it, it only predicts it 3.21% of the times when it represents 14.71% of the test set. Medium-Low also has a lower recall (0.175), similar to the Gaussian NB's; it is mostly classified as Low and Medium-High. Low is the class with the best performances, and still it was correctly classified only around 65% of the times. It is followed with similar, though slightly worse, performances by Medium-High, the most popular and most predicted class (around 43% of predictions). The **all-features DT** shows a dramatic improvement for the lowest performing class, High, especially for its recall (0.180), which finally surpasses the benchmark. The model has started to generalize on it: it predicts it twice as many times, especially for itself and its closest class, Medium-High. Medium-Low has also seen a great improvement in recall (0.319). Nevertheless, both classes get misclassified most of the time. Low has suffered a small decrease in recall in return (0.603), but remains the best performing class.

The 3 most important features for the numeric-only DT are *logRuntime*, *startYear* and *numVotes* (0.41, 0.29, 0.14, respectively), whereas for the all-features DT they are *tvEpisode*, *startYear* and *numVotes* (0.22, 0.14, 0.14, respectively): this is the only model where the runtime is not the most important feature. Looking at the first two levels of the DTs, we can see that the nodes are not very pure. For the numeric-only DT, the first split is on *logRuntime*<= 60.5 minutes manages to separate a significant part of the Medium-High and High classes (they are shorter) on the left subtree. For the all-features DT (Figure 34), the first split is on *tvEpisode*: the left subtree (not tvEpisode) separates the most part of the Low and Medium-Low classes, the subsequent split on movie further separates the High class: a large part of them are note tvEpisodes and are not movies.

4 REGRESSION

5 PATTERN MINING

In this section, we will use the FP-Growth algorithm to extract frequent itemsets and association rules. Preprocessing is required to turn the dataset into a list of transactions, with each record representing a transaction containing one item for each value the record takes for each feature. We have two multilabel variables, *genres* and *countryOfOrigin*, and we decided to only keep the four most frequent countries to reduce dimensionality, aggregating the rest into an 'Other' category. Numeric variables are discretized into quartile-based bins (as reported in Table 16), and, to allow interpretability, the name of the variables are appended to their values.

Variable	Bin 1	Bin 2	Bin 3	Bin 4
startYear	[1878, 1978]	(1978, 1997]	(1997, 2013]	(2013, 2024]
runMins.	[0, 29]	(29, 50]	(50, 90]	(90, 3000]
nVotes*	[1.792, 2.773]	(2.733, 3.611]	(3.611, 5.007]	(5.007, 13.782]
totImgs*	[0, 0.693]	(0.693, 1.946]	(1.946, 8.162]	–
totCreds*	[0, 2.833]	(2.833, 3.555]	(3.555, 4.19]	(4.19, 9.66]
numRegs*	(0.692, 1.386]	(1.386, 4.248]	–	–
critRevRat	[0, 0.227]	(0.227, 1]	–	–

Table 16: Bin intervals for each binned variable (*: log-transformed).

5.1 Frequent Itemsets Extraction

We keep the minimum number of items in the itemsets (*zmin*) fixed to two, as they could already be interesting. Figure 35 shows the number of closed and maximal frequent itemsets extracted for different *minsup* values, ranging from 2% to 30%. The number of itemsets drops quickly, and after 25% the two groups seem to start converg-

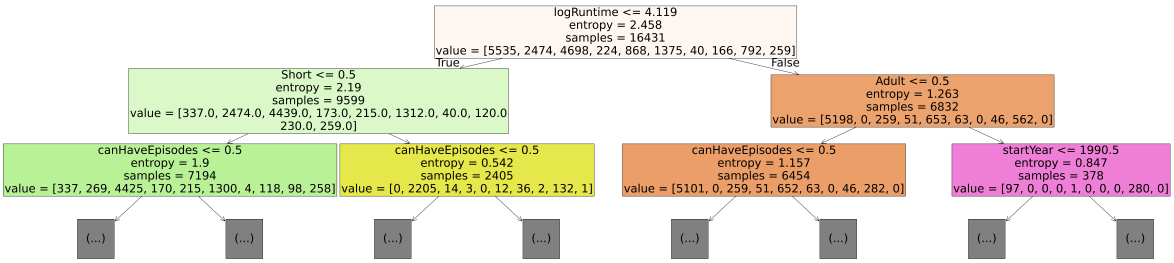

Figure 33: All-features DT for *titleType*, max_depth=2 (left= True, right= False).

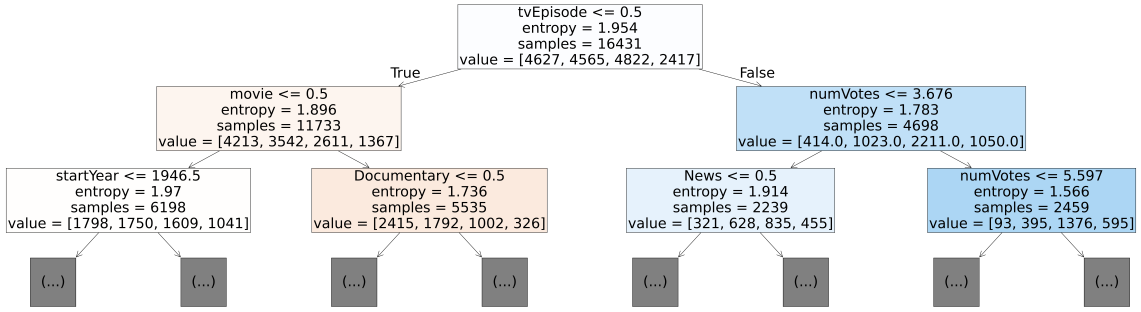

Figure 34: All-features DT for *binned rating*, max_depth=2 (left= True, right= False).

ing to lower values (< 250). We also look at the maximal frequent itemsets that contain our target variable, *titleType* (Figure 36): there are none for the minority classes tvShort, videoGame, tvMiniSeries, tvSpecial, and we start losing more classes from *minsup* > 4%.

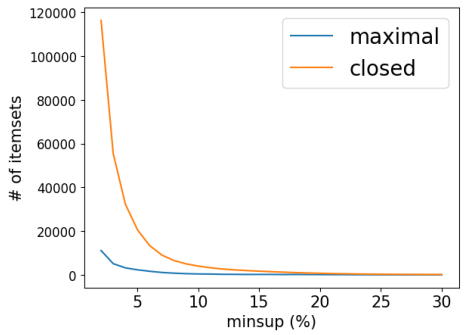

Figure 35: N. of closed and maximal frequent itemsets for different *minsup* values.

After testing different *minsup* values, we choose 30%, as we get some interesting maximal itemsets, rather than just a combination of the most frequent values for the different features. At this threshold, the frequent itemsets are 174 and they are all closed, while the maximal itemsets are only 26. In Table 17 are reported some of the most interesting maximal itemsets, among the top 10 with the highest support. The first one captures a dependency we expect: a movie doesn't have episodes. This is similarly captured by the 7th, as

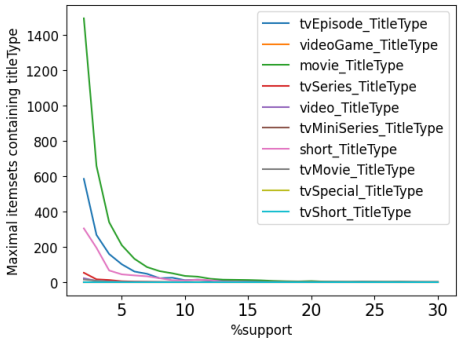

Figure 36: N. of maximal frequent itemsets containing *titleType*for different *minsup* values.

the reported runtime is typical of movies and tv episodes. The 4th one represents many foreign titles, made in just one foreign country, while the 6th captures dramatic stand-alone titles from a single country: this can well describe telenovela episodes, or lower budget dramatic movies (as they are made in a single country).

5.2 Association Rule Extraction

The heatmap in Figure 37 shows the number of association rules extracted by the FP-Growth algorithm for different *minsup* and confidence (*minconf*) thresholds. It is evident that the number of rules drops quickly as the *minsup* increases, while it is less affected by *minconf*. Therefore, we first

Rank	Maximal Itemset	Sup
1	(movie_TitleType, NoEpisodes)	33.686
4	(country_Other, NoVideos, OneCountryofOrigin)	32.481
6	(genre_Drama, NoEpisodes, OneCountryofOrigin)	31.514
7	((50.0, 90.0]_RuntimeMin, NoEpisodes)	31.471

Table 17: Selected top maximal itemsets for *minsup*=30.

kept a fixed low value of confidence (65%) and experimented with different values of *minsup*, increasing it by small increments. Once we found a value yielding an interesting list of rules, we increased the confidence threshold. We reached 80% *minconf* before losing interesting rules. The final hyperparameters are: *zmin*=2, *minsup*=9% and *minconf*=80%.

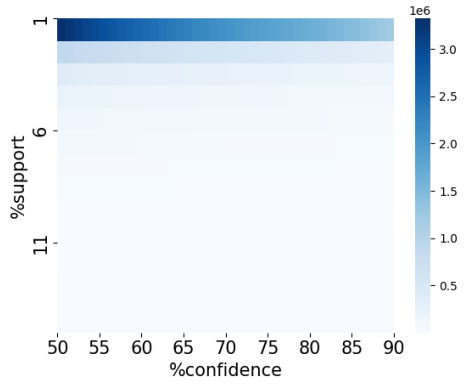

Figure 37: N. of association rules extracted for different *minsup* and *minconf* values.

The total number of rules extracted is 16466. The histograms in Figure 38 show the rules' confidence and lift distributions. Given the high threshold, rules have high confidence, with the majority having *conf* > 0.87. Nevertheless, the large majority is weak, with a lift < 1.

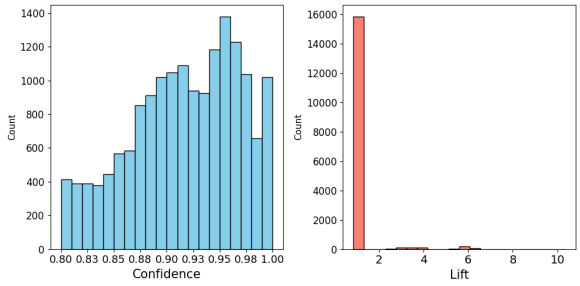

Figure 38: Histograms of rules' confidence and lift.

The 30 rules with the highest lift all contain a *titleType* either in the antecedent or consequent; a selection is reported in Table 18. In particular, the two top rules have captured the property of tvSeries of having an episode, while the rest have captured the redundancy of the information about shorts, present in both *genres* and *titleType*. In particular, rules having *short_titleType* as consequent are all associated with both *genre_Short* and the lowest value for *runtimeMinutes*, [0, 29].

Antecedent	Conseq.	%sup	conf	lift
(HasEpisodes, OneCountryofOrigin)	tvSeries _TitleType	7.89	0.866	10.35
(HasEpisodes)	tvSeries _TitleType	8.37	0.860	10.28
(short_TitleType, [0, 0.693]_TotalImages, NoEpisodes)	genre _Short	8.48	0.931	6.21
(genre_Short, [0, 29]_RuntimeMin, (0.692, 1.386]_NumRegions, NoEpisodes, OneCountryofOrigin)	short _TitleType	11.33	0.933	6.20
(short_TitleType, [0, 2.833]_TotalCredits, NoEpisodes)	genre _Short	8.58	0.928	6.19

Table 18: Selected top association rules by lift.

5.3 Classification: *titleType*

Out of the extracted rules, 245 have a *titleType* in the consequent: 144 have short, 89 have tvEpisode, 10 have movie, and two have tvSeries. For each of these four *titleType* classes, we implement the rule with the highest support for a task of multiclass classification, for the sake of which the remaining classes are aggregated to 'other'. We favor the rules with the highest support because they are more general and include the most important items in the antecedent; nevertheless, we make sure that their lift is higher than 1. The selected rules are reported in Table 19.

Antecedent	Conseq.	%sup	conf	lift
(genre_Short, NoEpisodes)	short _TitleType	13.42	0.900	5.980
((29.0, 50.0]_RuntimeMin, NoEpisodes)	tvEpisode _TitleType	16.57	0.873	3.054
((90.0, 3000.0]_RuntimeMin, NoEpisodes)	movie _TitleType	12.85	0.824	2.446
(HasEpisodes)	tvSeries _TitleType	8.37	0.860	10.276

Table 19: Association rules for *titleType* classification.

The performance of our simple rule-based classifier on the test set is reported in Table 20. It achieves an accuracy of 60% and a macro F1 score of 67.5%, outperforming our baseline model, a stratified dummy classifier trained on the training set, across all metrics for all classes (accuracy: 25.4%, macro F1: 20.4%).

	precision	recall	f1-score	support
movie	0.808	0.397	0.532	1877
other	0.210	0.607	0.312	789
short	0.890	0.911	0.901	766
tvEpisode	0.952	0.573	0.715	1599
tvSeries	0.847	1.000	0.917	447
accuracy			**0.600**	5478
macro avg	0.741	0.698	0.**675**	5478

Table 20: Classification report for rule-based classification.

The confusion matrix in Figure 39 allows us to understand what types of mistakes the classifier made and how the 'other' classes were classified. The information about having episodes was enough to achieve perfect recall for tvSeries, but of course all tvMiniSeries were also classified as such. The same happens for tvShorts, all classified as shorts – the same had happened for our other classifiers. The information about *runtimeMinutes* has lead to some misclassification, as the binning is quite coarse-grained; this explains the lower recall for tvEpisodes and, especially, for movies: there's plenty of movies with a runtime < 90 minutes.

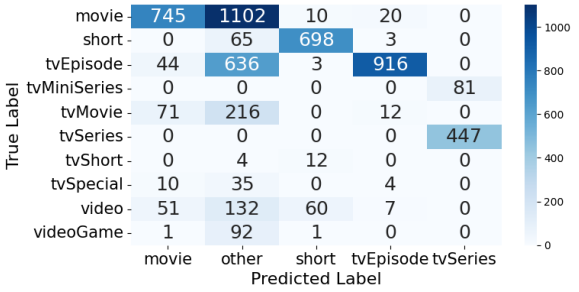

Figure 39: Confusion Matrix for rule-based classifier (10 True × 5 Predicted).

In conclusion, the association rules used leveraged on runtime, the information about serialized format, and the explicit information about shorts: these were exactly the top three most important features for our best performing DT model.